

Výstupy ŠVP**INF-09** Ověřím správnost, najdu a opravím případnou chybu v algoritmu, otestuji, odladím a optimalizuji program. (*Informatika 4, 2/2*)**Učivo:** testování, optimalizace – syntaktické, běhové a logické (funkční) chyby, krokování a ladění programu**Připomenutí**

Očekávaný výsledek si urči předem (ručně, zkouškou jako v matematice), teprve pak spustí program.

Sada testů: aspoň jeden test na každou větev + hraniční případy + nesmyslné vstupy (text, desetinná čárka).

Hlášku čti odspodu: poslední řádek = typ a popis chyby, nad ním místo; skutečná příčina může být i o řádek výš.

*Testování a ladění: testovací sada pro řešič rovnic, tři typy chyb, čtení chybových hlášek a krokování. Ke každému testu nejdřív napiš očekávaný výsledek. Podrobnosti najdeš v listu Must know.***A. Rozcvička**

- Urči typ chyby (syntaktická, běhová, logická): a) `print("Ahoj")`; b) `x = int("dvě")`; c) obvod obdélníku jako `obvod = 2 * a + b`; d) `if a = 0:`; e) `print(vysledek)`, když proměnná `vysledek` nevznikla; f) `math.sqrt(-4)`. ★☆☆
- Přečti hlášku. Na kterém řádku chyba nastala, jaký je to typ chyby a pro jaké vstupy mohla vzniknout? ★☆☆

```
Traceback (most recent call last):
  File "resic.py", line 4, in <module>
    x = -c / b
ZeroDivisionError: float division by zero
```

- Program obsahuje tři syntaktické chyby. Najdi je. Kterou z nich Python ohlásí a proč jen jednu? ★☆☆

```
a = float(input("a = "))
if a > 0
    print("kladné")
else:
    print("není kladné")
```

- Program má pro věk vypsát „dítě“ (0–14), „mladistvý“ (15–17) nebo „dospělý“ (18 a více). Navrhni testovací sadu aspoň 7 vstupů a u každého uveď očekávaný výstup a důvod výběru. ★★☆☆
- Program má spočítat cenu po slevě 20 %. Co vypíše? O jaký typ chyby jde? Oprav ho. ★☆☆

```
cena = 250
sleva = 20          # v procentech
po_sleve = cena - sleva / 100
print(po_sleve)
```

- Co se stane po spuštění? Oprav výpočet tak, aby test prošel. ★★☆☆

```
a, b = 3, 4
obvod = 2 * a + b
assert obvod == 14, "špatný obvod"
print("test prošel")
```

- Krokuj ručně: zapiš do trasovací tabulky hodnoty `x` a `y` po každém řádku. Co program vypíše a co dělá? ★☆☆

```
x = 3
y = 5
x = x + y
y = x - y
x = x - y
print(x, y)
```

- Uživatel zadá 2,5. Proč program spadne? Navrhni dvě možné opravy. ★☆☆

```
a = float(input("a = "))
print(2 * a)
```

B. Slovní úlohy

9. Sestav testovací sadu aspoň 8 testů pro svůj řešič kvadratické rovnice z H44 (vstup a, b, c – očekávaný výstup – co test ověřuje). Pokryj všech šest větví, záporné a a desetinná čísla. Spušť ji a zapiš skutečné výstupy. ★★☆☆
10. Který z testů ze sady odhalí chybu v tomto programu? Co program vypíše pro $x^2 + 4x + 4 = 0$? Oprav ho. ★★☆☆

```
import math
a, b, c = 1, 4, 4
D = b ** 2 - 4 * a * c
if D > 0:
    x1 = (-b + math.sqrt(D)) / (2 * a)
    x2 = (-b - math.sqrt(D)) / (2 * a)
    print(x1, x2)
else:
    print("nemá řešení")
```

11. Uprav načítání koeficientu tak, aby program při zadání „abc“ nespádl, ale vypsál upozornění. Vyzkoušej vstupy 3.5 a abc. ★★☆☆

```
try:
    a = float(input("a = "))
    print("Načteno:", a)
except ValueError:
    print("Zadej číslo (desetinnou tečkou).")
```

12. V Python Tutor (v prohlížeči) nebo v Thonny (Debug, krok přes) krokuj opravený program z úlohy 10. Zapiš, které řádky se provedou pro vstup (1, 4, 4) a které pro (1, -5, 6), a jakou hodnotu má D. ★☆☆
13. Podle zadání platí: děti 6–15 let a senioři od 65 let platí 60 Kč, děti do 6 let nic, ostatní 120 Kč. Který test chybu odhalí? Navrhni hraniční testy a program oprav. ★★☆☆

```
vek = int(input("Věk: "))
if vek < 6:
    cena = 0
elif vek < 15 or vek > 65:
    cena = 60
else:
    cena = 120
print(cena)
```

14. Optimalizace: Program počítá stejný výraz třikrát. Uprav ho, aby diskriminant i odmocninu spočítal jen jednou. Ověř, že výstup zůstal stejný. ★☆☆

```
import math
a, b, c = 2, -4, -6
if b ** 2 - 4 * a * c > 0:
    print((-b + math.sqrt(b ** 2 - 4 * a * c)) / (2 * a))
    print((-b - math.sqrt(b ** 2 - 4 * a * c)) / (2 * a))
```

15. Zkouška = test (H46): Rovnici $\sqrt{x+2} = x$ umocníme na $x^2 - x - 2 = 0$ s kořeny 2 a -1. Doplníš do programu „zkoušku“, která dosadí kořen do původní rovnice a vypíše jen vyhovující kořeny. Jakou složenou podmínku použiješ? ★★☆☆
16. Napiš záznam ladění k úloze 13 v pěti bodech: co jsem čekal – co se stalo – hypotéza (proč) – jak jsem ji ověřil – oprava a opakovaný test. ★☆☆

C. Problémy a propojení

17. Kolik testů by bylo potřeba, kdybychom řešič zkoušeli pro všechna celá a, b, c od -10 do 10? Proč místo toho volíme po jednom zástupci každé větve a hranice? Jak by šlo všechny kombinace otestovat automaticky (náhled na H49)? ★★☆☆
18. Přestupný rok zjišťuje program jen podmínkou $r \% 4 == 0$. Testy 2020, 2023, 2024, 2025 projdou. Který test chybu odhalí? Ukaž, že pro všechny roky 1901–2099 dává chybný program správnou odpověď. Co to říká o testování? ★★☆☆
19. Řešič $ax + b = 0$ vypíše pro $a = 0,1$, $b = -0,3$ výsledek 2.9999999999999996. Je to chyba programu? Jak má test takový výsledek vyhodnotit a jak ho vypsát uživateli? ★★☆☆

20. Reálná situace: a) Raketa Ariane 5 (1996) se krátce po startu zničila: software převzatý z Ariane 4 převáděl 64bitové desetinné číslo (horizontální rychlost) na 16bitové celé číslo a pro větší hodnoty nové rakety převod selhal. b) Sonda Mars Climate Orbiter (1999) zanikla, protože jeden program počítal v librách-síla krát sekunda a druhý očekával newtony krát sekunda. Urči typ chyby a navrhní test, který by ji odhalil. ★★★

Hotové úlohy k procvičení na internetu

- **Nauč se Python (PyLadies) – Print a chybové hlášky** – jak číst chybovou hlášku (soubor, řádek, typ chyby)
<https://naucese.python.cz/course/pyladies/beginners/print/>
- **Nauč se Python (PyLadies) – Výjimky** – ValueError, ZeroDivisionError a ošetření vstupu pomocí try/except
<https://naucese.python.cz/course/pyladies/beginners/exceptions/>
- **Nauč se Python (PyLadies) – Testování** – automatické testy pomocí assert a pytest (pro zájemce)
<https://naucese.python.cz/course/pyladies/beginners/testing/>

★ základní · ★★ střední · ★★★ náročnější. Vyber si, které úlohy spočítáš; řešení ukládej do svého portfolia.

Výsledky ke kontrole

1. a) syntaktická (neukončený text); b) běhová ValueError; c) logická (má být $2 * (a + b)$); d) syntaktická (porovnání je ==); e) běhová NameError; f) běhová ValueError: math domain error
2. řádek 4, ZeroDivisionError (dělení nulou) – $b = 0$, tedy vstup s $a = 0$ a $b = 0$ (např. 0, 0, 5); chybí větev pro $b = 0$
3. ř. 1 chybí „“, ř. 2 chybí „:“, ř. 5 chybí odsazení; Python skončí u první nalezené chyby (novější verze: „(' was never closed“ na ř. 1, starší „invalid syntax“ až o řádek či dva níž) – po opravě hlásí další
4. např. 0 (dítě, dolní hranice), 14 (dítě, hranice), 15 (mladistvý), 17 (mladistvý), 18 (dospělý), 45 (běžný případ), -1 (neplatný), „abc“ (nečíselný vstup → upozornění)
5. 249.8 místo 200 – logická chyba; oprava: $po_sleva = cena * (1 - sleva / 100)$ → 200.0
6. AssertionError: špatný obvod (obvod je 10); oprava $obvod = 2 * (a + b)$ → vypíše test prošel
7. (x, y): (3, -), (3, 5), (8, 5), (8, 3), (5, 3); vypíše 5 3 – prohodí hodnoty bez pomocné proměnné
8. ValueError: could not convert string to float: '2,5' – Python chce desetinnou tečku; opravy: $float(input().replace(",", "."))$ nebo try/except s upozorněním
9. např. (1, -5, 6) → 3, 2; (1, 2, 1) → -1 ($D = 0$); (1, 0, 1) → nemá řešení; (0, 2, -4) → 2; (0, 0, 0) → každé číslo; (0, 0, 3) → žádné; (-1, 0, 4) → -2 a 2 (záporné a); (1, 0.2, 0.01) → -0,1 (tolerance u $D = 0$)
10. vypíše „nemá řešení“, správně $x = -2$ – chybí větev $D = 0$; odhalí ji jen hraniční test s $D = 0$; oprava: $elif D == 0: print(-b / (2 * a))$ → -2.0
11. 3.5 → Načteno: 3.5; abc → Zadej číslo (desetinnou tečku). – blok except zachytí ValueError
12. (1, 4, 4): řádky 1–4, test $D > 0$ neplatí, provede se větev $elif D == 0$ ($D = 0$) a výpis -2.0; (1, -5, 6): $D = 1$, provede se větev $D > 0$ s výpočtem $x_1, x_2 \rightarrow 3.0 \ 2.0$
13. chybné hranice: 15 → vypíše 120 (má 60), 65 → vypíše 120 (má 60); hraniční testy 5, 6, 15, 16, 64, 65; oprava $vek \leq 15$ or $vek \geq 65$
14. $D = b^2 - 4 * a * c$; if $D > 0$: $s = \text{math.sqrt}(D)$, pak $(-b + s) / (2 * a)$ a $(-b - s) / (2 * a)$; výstup v obou verzích 3.0 a -1.0 – kratší, čitelnější a méně výpočtů
15. podmínka $x + 2 \geq 0$ and $\text{abs}(\text{math.sqrt}(x + 2) - x) < 1e-9$; 2 vyhovuje ($\sqrt{4} = 2$), -1 nevyhovuje ($\sqrt{1} = 1 \neq -1$) – umocnění není ekvivalentní úprava
16. čekal jsem 60 Kč pro věk 15; program vypsál 120; hypotéza: špatně zapsaná hranice ($<$ místo $\leq$); ověření: krokování ukázalo, že $vek < 15$ je pro 15 False; oprava $vek \leq 15$ or $vek \geq 65$, celá sada znovu – vše prošlo
17. $21^3 = 9\ 261$ kombinací – ručně nereálné; vstupy se stejnou větví se chovají stejně, stačí zástupce + hranice; automaticky: cykly přes všechna a, b, c a kontrola dosazením kořene do rovnice (assert)
18. odhalí ji rok 1900 nebo 2100 (dělitelné 100, ne 400); v letech 1901–2099 je jediný rok dělitelný 100 rok 2000, a ten je dělitelný i 400 → obě podmínky se shodují; testy jen z „běžného“ období chybu nikdy nenajdou – testuj hranice pravidla, ne jen obvyklá data
19. není to chyba algoritmu, ale zaokrouhlení v binárním float (0,1 a 0,3 nejsou přesné); test porovnává s tolerancí $\text{abs}(x - 3) < 1e-9$; uživateli vypsát $\text{round}(x, 6) \rightarrow 3.0$
20. a) běhová chyba (přetečení při převodu, neošetřená výjimka) – test s hodnotami z nové, rychlejší dráhy, tj. mimo rozsah starého systému (hraniční/velké vstupy); b) logická chyba rozhraní (jednotky) – test spolupráce obou programů a porovnání s nezávislým výpočtem; poučení: jednotky a rozsahy patří do dokumentace a testů

Řešení krok za krokem

Úloha 18

1. Pravidlo: rok je přestupný, je-li dělitelný 4, ale ne 100 – s výjimkou roků dělitelných 400.
2. Podmínka $r \% 4 == 0$ se od pravidla liší jen u roků dělitelných 100, ale ne 400 (1900, 2100, 2200...).
3. Testy 2020, 2023, 2024 a 2025 takový rok neobsahují, proto projdou; chybu odhalí test 1900 nebo 2100 (očekáváno: nepřestupný).
4. V letech 1901–2099 je jediný rok dělitelný 100 rok 2000 – a ten je dělitelný i 400, je tedy přestupný podle obou pravidel.
5. Pro všechny roky z tohoto období dávají obě podmínky stejnou odpověď, chybný program proto odpovídá správně.
6. Poučení: testy z „běžného“ období chybu nenajdou – sada musí obsahovat hranice a výjimky pravidla.

Úloha 20

1. a) Ariane 5: program za letu spadl – převod na 16bitové celé číslo (nejvýš 32 767) přetekl a výjimka nebyla ošetřena → běhová chyba.
2. Příčina: kód převzatý z Ariane 4 počítal s rozsahem hodnot, který nová, rychlejší raketa překročila.
3. Test: simulace s daty z dráhy Ariane 5, tj. hraniční a větší vstupy než u staré rakety, a kontrola, co program udělá při přetečení.
4. b) Mars Climate Orbiter: programy běžely bez pádu, ale s chybnými čísly – logická chyba na rozhraní (jednotky; $1 \text{ lbf} \cdot \text{s} \approx 4,45 \text{ N} \cdot \text{s}$).
5. Test: propojit oba programy a výstup porovnat s nezávislým výpočtem v jednotkách SI (test rozhraní).
6. Poučení: rozsahy hodnot a jednotky patří do dokumentace i do testů.